Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

По дисциплине «Интеллектуальный анализ данных»

Тема: «PCA»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Данилюк Т.В.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование numpy.linalg.eig для вычисления собственных значений и собственных векторов и 2. с помощью sklearn.decomposition.PCA для непосредственного применения метода PCA – два независимых варианта решения);

2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки matplotlib, обозначая экземпляры разных классов с использованием разных цветовых маркеров;

3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;

4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Класс
2	hcv+data.zip	Category

Ход работы

Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# ---------- 1. Загрузка данных ----------
columns = [
    "ID", "Category", "Age", "Sex", "ALB", "ALP",
    "ALT", "AST", "BIL", "CHE", "CHOL", "CREA",
    "GGT", "PROT"
]

# Файл hcvdat0.csv из архива hcv+data.zip (разделитель – запятая, есть заголовок)
data = pd.read_csv("hcvdat0.csv", sep=",", header=0)

# Приводим имена столбцов к нужным (на случай лишних пробелов)
data.columns = [c.strip() for c in data.columns]

# ---------- 2. Признаки и метка класса ----------
features = [
    "Age", "ALB", "ALP", "ALT", "AST",
    "BIL", "CHE", "CHOL", "CREA", "GGT", "PROT"
]

X = data[features].apply(pd.to_numeric, errors="coerce")
y = data["Category"].astype(str).str.strip()

# ---------- 3. Обработка пропусков (стратегия: заполнение средним) ----------
X = X.fillna(X.mean())

print("Размер исходных данных:", X.shape)

# ---------- 4. Стандартизация (важно для PCA) ----------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ---------- 5. PCA вручную через numpy.linalg.eig ----------
# Ковариационная матрица
cov_matrix = np.cov(X_scaled, rowvar=False)

# Собственные значения и собственные векторы
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Сортировка по убыванию собственных значений
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
```

```
eigenvectors = eigenvectors[:, idx]

# Действительная часть (на случай комплексных чисел)
eigenvalues = np.real(eigenvalues)
eigenvectors = np.real(eigenvectors)

print("Собственные значения:", eigenvalues)

# Доля объяснённой дисперсии
explained_variance_ratio = eigenvalues / np.sum(eigenvalues)
print("Доля объяснённой дисперсии:", explained_variance_ratio)

# Проекция на 3 главные компоненты (вручную)
X_pca_manual = X_scaled.dot(eigenvectors[:, :3])
print("Размерность после PCA (manual):", X_pca_manual.shape)

# ---------- 6. PCA через sklearn ----------
pca = PCA(n_components=3)
X_pca_sklearn = pca.fit_transform(X_scaled)

print("PCA sklearn: Собственные значения:", pca.explained_variance_)
print("Доля объяснённой дисперсии:", pca.explained_variance_ratio_)

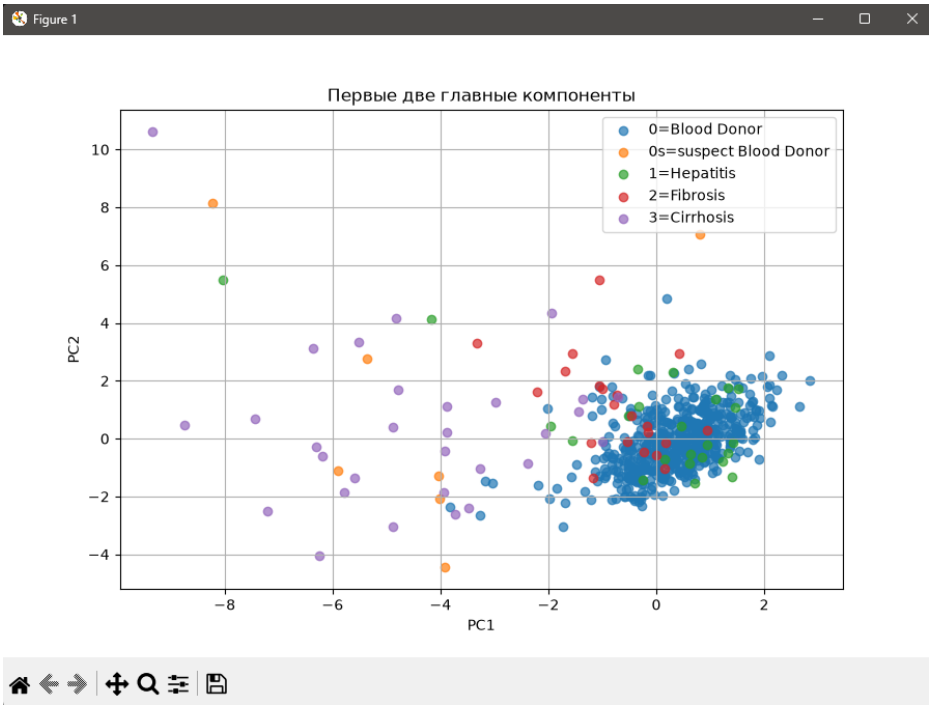
# ---------- 7. Потери при использовании 2 и 3 компонент ----------
loss_2 = 1 - np.sum(explained_variance_ratio[:2])
loss_3 = 1 - np.sum(explained_variance_ratio[:3])
print("Потери при использовании 2 компонент:", loss_2)
print("Потери при использовании 3 компонент:", loss_3)

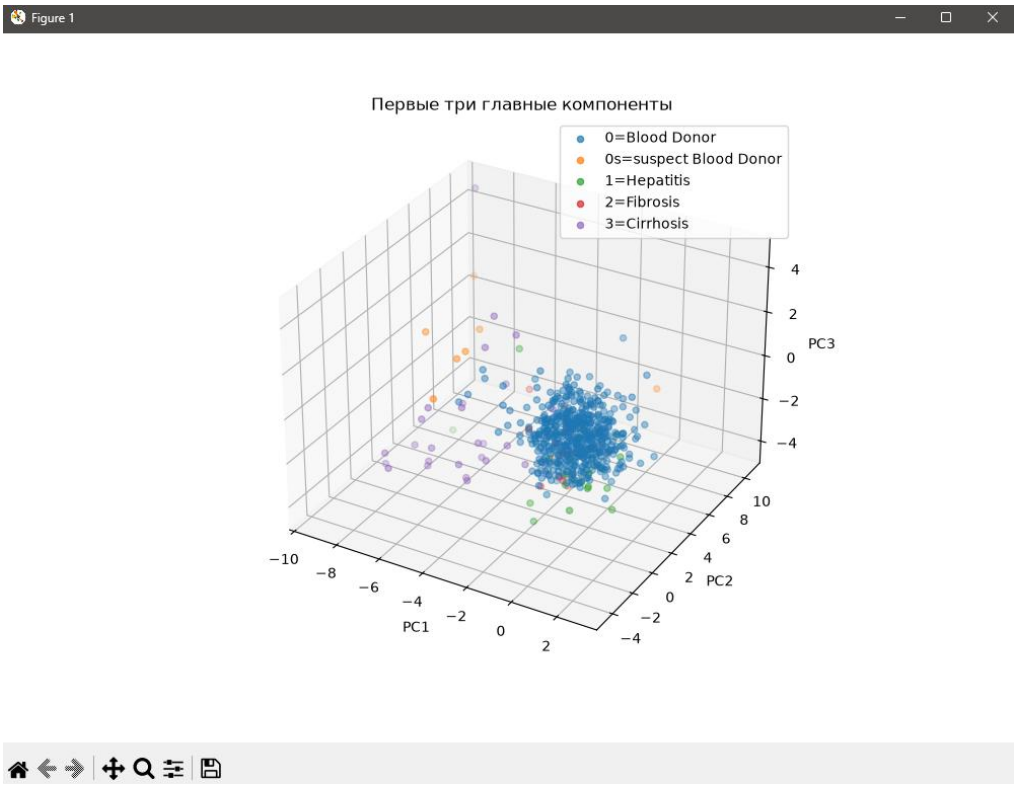
# ---------- 8. Визуализация ----------
classes = sorted(y.unique())

# 2D — первые две главные компоненты
plt.figure(figsize=(9, 6))
for cls in classes:
    mask = y == cls
    plt.scatter(
        X_pca_sklearn[mask, 0],
        X_pca_sklearn[mask, 1],
        label=cls,
        alpha=0.7
    )
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Первые две главные компоненты")
plt.legend()
plt.grid()
plt.show()

# 3D — первые три главные компоненты
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection="3d")
for cls in classes:
    mask = y == cls
    ax.scatter(
        X_pca_sklearn[mask, 0],
        X_pca_sklearn[mask, 1],
        X_pca_sklearn[mask, 2],
        label=cls,
        alpha=0.7
    )
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("Первые три главные компоненты")
ax.legend()
plt.show()
```

Графики:





Вывод программы:

```
Размер исходных данных: (615, 11)
Собственные значения: [2.46688166 1.82434186 1.36515764 1.09635732 0.96536308 0.76162644
 0.70126918 0.58938488 0.49366308 0.42025831 0.33361187]
Доля объяснённой дисперсии: [0.22389732 0.16557959 0.12390344 0.09950678 0.08761758 0.06912618
 0.06364808 0.05349332 0.04480549 0.03814318 0.03027904]
Размерность после PCA (manual): (615, 3)
PCA sklearn: Собственные значения: [2.46688166 1.82434186 1.36515764]
Доля объяснённой дисперсии: [0.22389732 0.16557959 0.12390344]
Потери при использовании 2 компонент: 0.6105230976173619
Потери при использовании 3 компонент: 0.4866196549498729

Process finished with exit code 0
```

Первые две главные компоненты сохраняют 38,95% дисперсии исходных данных, при этом потери информации составляют 61,05%. При использовании трёх главных компонент сохраняется уже 51,13% дисперсии, а потери снижаются до 48,69%.

Таким образом, увеличение количества главных компонент с двух до трёх позволяет сохранить больше информации об исходных данных. Полученные графики позволяют визуально представить распределение объектов выборки в пространстве главных компонент.

Вывод: научился применять метод PCA для осуществления визуализации данных.